

PantawidAral – System Design Document

Version: 1.0 Status: Hackathon prototype specification Document purpose: Technical blueprint for 6-hour build. Drop into Cursor / Claude Code as single source of truth.

1. Document Overview

1.1 Purpose

This document specifies the technical design of PantawidAral, a dropout risk prediction and intervention support tool for DSWD social workers managing 4Ps-enrolled children. The document is the binding reference for the hackathon build and should be sufficient for an AI coding assistant to scaffold the system without further specification.

1.2 Scope

The hackathon build delivers a vertical slice of the full system: one social worker persona, one cluster of synthetic 4Ps families, end-to-end prediction pipeline, plain-language case note generation, and the ethical access boundary made visible. Out of scope: production data integration, real auth, multi-tenant deployment, mobile clients.

1.3 Audience

Hackathon team developers, AI coding assistants used during the build, judges reviewing technical execution post-pitch.

2. System Architecture

2.1 Architectural Style

Monolithic full-stack web application with a clear three-layer separation:

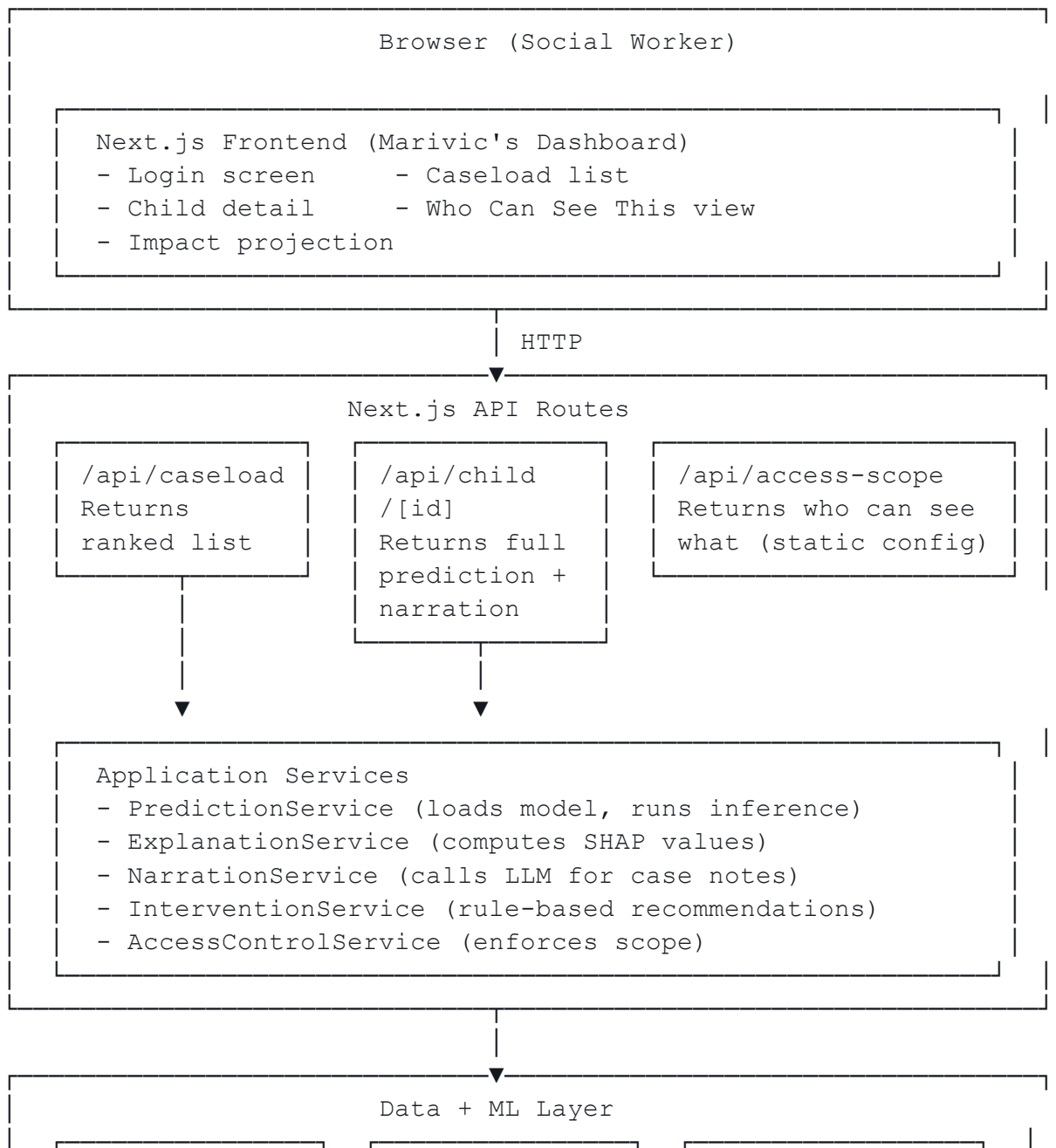
- Presentation layer – Next.js 14 frontend (App Router, TypeScript, Tailwind)
- Application layer – Next.js API routes orchestrating prediction, narration, and access control
- Data + ML layer – Static synthetic dataset + pre-trained model artifact + LLM API for narration

No microservices. No database. No auth provider. The system is a single deployable unit.

2.2 Why This Architecture

The rubric rewards technical execution that is functional end-to-end and appropriate. A monolith here is appropriate: one team, six hours, one demo. Microservices would be overengineered. The three-layer separation gives us defensible structure without cost. The static synthetic dataset choice is critical: real DSWD data is not legally accessible for a hackathon, and a synthetic dataset designed to reflect documented Philippine dropout patterns is more defensible than scraped or fake-real data.

2.3 High-Level Component Diagram



synthetic_ caseload.json (247 families)	model.pkl shap_ explainer.pkl	Anthropic API (case note generation)
---	-------------------------------------	--

2.4 Request Flow (Critical Path)

When the social worker clicks on a flagged child:

1. Frontend calls `GET /api/child/{id}`
2. API route invokes `AccessControlService.verify(role='social_worker', resource=childId)`
3. On verification, `PredictionService.predict(childFeatures)` returns probability + features
4. `ExplanationService.explain(prediction)` returns top-N SHAP contributors
5. `NarrationService.narrate(features, shapValues)` calls the LLM for plain-language case note (with caching for demo personas)
6. `InterventionService.recommend(shapValues)` returns rule-based intervention list
7. API assembles response, returns to frontend
8. Frontend renders child detail view

Total expected latency: 2–6 seconds, dominated by the LLM call. Pre-cached for demo personas to be near-instant.

3. Technology Stack

3.1 Stack Decisions

Layer	Choice	Rationale
Frontend framework	Next.js 14 (App Router)	Single project for FE+API; fast deploy to Vercel
Language (FE)	TypeScript	Type safety pays for itself in 6h; reduces runtime bugs
Styling	TailwindCSS	No design system bikeshedding; fast iteration

UI components	shadcn/ui	Pre-built accessible components; copy-paste, no dependency hell
Backend	Next.js API routes (Node)	Same project, same deploy
ML training	Python 3.11 + scikit-learn + LightGBM + SHAP	Industry standard; offline training, model artifact loaded at runtime
ML inference	ONNX Runtime (via <code>onnxruntime-node</code>) OR Python microservice via subprocess	Allows trained Python model to be called from Node API route
LLM (narration)	Anthropic Claude (claude-sonnet-4-5)	Best-in-class structured output; handles Tagalog mix gracefully
Data store	Static JSON files in <code>/data</code>	No database; everything load-once-at-startup
Hosting	Vercel	One-command deploy; team has likely used it
Charts	Recharts (in React)	Calibration plot, fairness audit visualizations

3.2 Critical Technical Decision: Python Model in Node API

Problem: LightGBM models are easiest to train in Python with SHAP. But our API runs in Node.

Solution: Train in Python, export model to ONNX format, load via `onnxruntime-node` in the API route. SHAP values are pre-computed offline for all synthetic caseload children and stored alongside the dataset (since the caseload is static, this is sound).

Fallback if ONNX export fails: Use `python-shell` to call a small Python inference script as subprocess. Slower but works. Decision made by hour 2.

3.3 Required Environment Variables

env

```
ANTHROPIC_API_KEY=sk-ant-...
DEMO_MODE=true # Enables pre-cached responses for
personas
NODE_ENV=production
```

4. Project Structure

```
pantawidaral/
├── app/ # Next.js App Router
│   └── page.tsx # Landing - "Login as Ate Marivic"
├── dashboard/
│   └── page.tsx # Caseload view (12 flagged children)
├── child/
│   └── [id]/page.tsx # Child detail with case note
├── access/
│   └── page.tsx # "Who Can See This" ethical screen
├── impact/
│   └── page.tsx # Cluster-level impact projection
├── api/
│   ├── caseload/route.ts # GET ranked caseload
│   └── child/[id]/route.ts # GET full prediction + narration
├── access-scope/route.ts # GET access control config
├── impact/route.ts # GET cluster-level projection
├── components/
│   ├── CaseloadTable.tsx
│   ├── ChildDetailCard.tsx
│   ├── RiskBadge.tsx
│   ├── CaseNote.tsx
│   ├── InterventionList.tsx
│   └── AccessScopeMatrix.tsx # Who Can See This
├── visualization
│   ├── ImpactProjection.tsx
│   └── EthicsBanner.tsx # Persistent ethics indicator
├── lib/
└── services/
```

```

|   |   | prediction.ts           # Model loading + inference
|   |   | explanation.ts         # SHAP value retrieval
|   |   | narration.ts           # LLM-based case note
generation
|   |   | intervention.ts        # Rule-based recommendations
|   |   | accessControl.ts       # Scope enforcement
|   |   | types.ts               # TypeScript interfaces
(canonical)
|   |   | prompts.ts             # LLM prompt templates
|   |   | constants.ts           # Risk thresholds, feature
lists
|
| data/
|   | synthetic_caseload.json     # 247 families with full
features
|   | precomputed_predictions.json # Pre-run predictions + SHAP
|   | precomputed_narrations.json # Pre-cached LLM case notes
|   | intervention_rules.json     # Risk-driver to intervention
mapping
|   | access_scope.json           # Who can see what
|   | impact_baseline.json        # DSWD-cited baseline numbers
|
| ml/                             # Offline training (run before
hackathon)
|   | generate_synthetic.py        # Generate 247-family dataset
|   | train_model.py              # Train LightGBM, export ONNX
|   | compute_shap.py             # Pre-compute SHAP for all
children
|   | pregenerate_narrations.py    # Pre-call LLM for demo
children
|   | validate_fairness.py         # Run fairness audit
|
| public/
|   | personas/
|       | marivic.jpg             # Optional persona avatar
|
| .env.local
| package.json
| tsconfig.json
| tailwind.config.js
| README.md

```

5. Data Model

5.1 Canonical Types

typescript

```
// lib/types.ts
```

```
export interface Child {
  id: string; // "child_0001"
  firstName: string; // "Mark"
  age: number; // 14
  gradeLevel: string; // "Grade 8"
  schoolName: string; // "San Pedro Elementary"
  fpsHouseholdId: string; // "4PS-LAG-00247"
  features: ChildFeatures;
  guardianName: string; // For social worker reference
  only
  barangay: string; // "Barangay San Roque"
}

export interface ChildFeatures {
  // Attendance
  attendanceRate30d: number; // 0.71 (last 30 days)
  attendanceRate90d: number; // 0.84
  attendanceTrend: number; // -0.25 (slope; negative =
declining)

  // Academic
  averageGrade: number; // 78.5
  gradeTrend: number; // -3.2 (per quarter)
  failingSubjects: number; // 2

  // Household
  householdIncomeShock: boolean; // Reported in last 6 months
  parentEmploymentChange: boolean;
  householdSize: number;
  numberOfSiblings: number;
  hasOlderSiblingDropout: boolean;

  // Demographic / Structural
  ageGradeMismatch: number; // 0 = on-track, +N = over-age
  distanceToSchoolKm: number;
  recentRelocation: boolean;

  // 4Ps Compliance
  monthsIn4Ps: number;
  recentComplianceWarnings: number;
```

```

}

export interface Prediction {
  childId: string;
  dropoutProbability90d: number;           // 0.73
  riskTier: 'low' | 'moderate' | 'high' | 'critical';
  topRiskDrivers: RiskDriver[];           // Top 3-5 SHAP contributors
  confidence: number;                     // Model's calibrated
confidence
  generatedAt: string;                     // ISO timestamp
}

export interface RiskDriver {
  feature: string;                         // "attendanceRate30d"
  humanLabel: string;                     // "Attendance has dropped
sharply"
  shapValue: number;                       // +0.18
  contributionDirection: 'increases' | 'decreases';
}

export interface CaseNote {
  childId: string;
  narrativeText: string;                   // LLM-generated, social
worker tone
  generatedAt: string;
  source: 'live' | 'cached';               // Demo transparency
}

export interface Intervention {
  type: 'home_visit' | 'cash_assistance' | 'academic_support'
    | 'health_referral' | 'family_counseling' |
'school_coordination';
  urgencyDays: number;                     // 7 = "within 7 days"
  description: string;
  rationale: string;                       // Which risk driver this
addresses
  dswdProgramReference?: string;           // e.g. "DSWD AICS"
}

export interface AccessScope {
  role: string;                           // "Social Worker"
  canSee: string[];                       // ["risk_score", "case_note",
...]
  cannotSee: string[];
  rationale: string;

```



```

}

export interface ImpactProjection {
  clusterSize: number; // 247 families
  flaggedThisWeek: number; // 12 children
  projectedDropoutsWithoutIntervention: number;
  projectedDropoutsWithIntervention: number;
  preventionRate: number; // 0.58
  citationSources: string[]; // DSWD/DepEd reports
}

```

5.2 Synthetic Dataset Specification

`data/synthetic_caseload.json` contains 247 children in Ate Marivic's cluster, generated with these distribution targets:

- Risk distribution: 70% low, 18% moderate, 8% high, 4% critical
- Demographic spread: Age 6–17, balanced gender, distributed across 8 barangays
- Realistic co-occurrence patterns: Income shocks correlate with attendance drops; older sibling dropouts correlate with current child risk; relocation correlates with grade drops

Three "demo children" are hand-tuned for the pitch:

1. Mark, 14, critical risk (0.73) — attendance crash + income shock + sibling dropout (the headline demo)
2. Sofia, 11, high risk (0.58) — academic decline + relocation + age-grade mismatch
3. Joshua, 16, moderate risk (0.41) — early warning, lighter signals (shows the system catches subtle cases too)

These three have pre-cached LLM narrations and pre-computed SHAP values for guaranteed demo reliability.

6. ML Pipeline Specification

6.1 Training Pipeline (Offline, Pre-Hackathon)

python

```
# ml/train_model.py — pseudocode
```

1. Load `synthetic_caseload.json` into pandas DataFrame
2. Generate synthetic outcome labels using rule-based logic:
 - High attendance trend negative → high dropout probability
 - Income shock + sibling dropout → high dropout probability
 - Add realistic noise (~15% label flips)
3. Train/test split (80/20, stratified by risk tier)
4. Train LightGBM classifier:

```
- 200 estimators, max_depth=6, learning_rate=0.05
- Class weights balanced
5. Calibrate using sklearn's CalibratedClassifierCV (isotonic)
6. Compute metrics: AUC-ROC, calibration curve, fairness across
groups
7. Export model to ONNX format → model.onnx
8. Save scikit-learn pipeline → model.pkl (fallback)
```

Target metrics on synthetic test set:

- AUC-ROC ≥ 0.85
- Calibration ECE ≤ 0.05
- Fairness disparity (TPR across gender, region) ≤ 0.10

These are documented in the pitch as "evaluated on held-out synthetic data; production deployment would require re-validation on real DSWD data."

6.2 SHAP Pre-Computation

python

```
# ml/compute_shap.py - pseudocode
```

```
1. Load model.pkl
2. Load synthetic_caseload.json
3. For each child:
    - Compute SHAP values via TreeExplainer
    - Extract top 5 features by absolute SHAP value
    - Map to human labels (see Section 6.4)
4. Save to data/precomputed_predictions.json
```

This avoids running SHAP at inference time, which is computationally expensive and would slow down the demo.

6.3 Feature-to-Human-Label Mapping

python

```
# Used in compute_shap.py
```

```
FEATURE_LABELS = {
    "attendanceRate30d": {
        "increases": "Attendance has dropped sharply in the last month",
        "decreases": "Attendance has remained strong"
    },
    "householdIncomeShock": {
        "increases": "The household reported a recent income loss",
        "decreases": "Household income has been stable"
    },
}
```

```

    "hasOlderSiblingDropout": {
      "increases": "An older sibling has previously left school",
      "decreases": "All older siblings completed schooling"
    },
    "ageGradeMismatch": {
      "increases": "The child is older than typical for their grade level",
      "decreases": "The child is on-track for their age"
    },
    // ... full mapping for all features
  }
}

```

6.4 Fairness Audit (Documented in Pitch)

python

```
# ml/validate_fairness.py - produces fairness_report.json
```

Compute and report:

- True positive rate by gender, by region
- False positive rate by gender, by region
- Calibration by subgroup
- Demographic parity ratios

The fairness report is shown in the pitch as a single chart proving the model performs equitably across protected groups. This is the kind of detail that converts a 4 to a 5 on Technical Execution.

7. API Specification

7.1 GET /api/caseload

Auth: Demo-mode authenticated as Ate Marivic Returns: Ranked list of children in cluster

Response (200):

json

```

{
  "socialWorker": {
    "name": "Marivic Santos",
    "role": "Municipal Link",
    "cluster": "San Pedro, Laguna",
    "totalFamilies": 247
  },
  "children": [
    {
      "child": { /* Child object, abbreviated */ },

```

```

        "prediction": { /* Prediction object */ },
        "interventionScheduled": false
    },
    // ... sorted by dropoutProbability90d desc
],
"summary": {
    "totalFlagged": 12,
    "criticalRisk": 3,
    "highRisk": 5,
    "moderateRisk": 4
}
}

```

7.2 GET /api/child/[id]

Returns: Full child detail including case note and interventions

Response (200):

json

```

{
  "child": { /* Child */ },
  "prediction": { /* Prediction */ },
  "caseNote": { /* CaseNote */ },
  "interventions": [ /* Intervention[] */ ],
  "accessAuditLog": [
    {
      "accessedBy": "Marivic Santos",
      "role": "Municipal Link",
      "timestamp": "2026-05-07T08:23:11Z"
    }
  ]
}

```

Behavior:

1. If `id` is in `precomputed_narrations.json` AND `DEMO_MODE=true`, return cached version (with `caseNote.source: "cached"`)
2. Otherwise, generate live narration via LLM
3. Always log access to audit trail (in-memory for demo; would be persisted in production)

7.3 GET /api/access-scope

Returns: Static configuration showing who can see what

Response (200):

json

```

{

```

```

"matrix": [
  {
    "role": "Social Worker (Municipal Link)",
    "canSee": ["risk_score", "case_note", "interventions",
"household_data"],
    "cannotSee": [],
    "rationale": "Direct intervention responsibility"
  },
  {
    "role": "School Teacher",
    "canSee": [],
    "cannotSee": ["risk_score", "case_note", "interventions",
"household_data"],
    "rationale": "Avoiding labeling effects documented in education
research"
  },
  {
    "role": "School Administrator",
    "canSee": [],
    "cannotSee": ["risk_score", "case_note", "interventions",
"household_data"],
    "rationale": "No direct intervention mandate; risk of misuse"
  },
  {
    "role": "Family",
    "canSee": ["case_note (with consent)", "interventions"],
    "cannotSee": ["raw risk_score"],
    "rationale": "Collaborative discussion via social worker, not
raw score"
  },
  {
    "role": "DSWD Case Management",
    "canSee": ["risk_score", "case_note", "aggregate_statistics"],
    "cannotSee": ["individually_identifying_household_data"],
    "rationale": "Program oversight; data minimization"
  },
  {
    "role": "External Researchers",
    "canSee": ["aggregate_statistics_only"],
    "cannotSee": ["individual_predictions",
"personally_identifying_data"],
    "rationale": "Research access is anonymized only"
  }
],

```

```
    "principle": "The system is built to be useful only to those who  
can help, and useless to those who could harm."  
}
```

7.4 GET /api/impact

Returns: Cluster-level impact projection

Response (200):

json

```
{  
  "projection": {  
    "clusterSize": 247,  
    "flaggedThisWeek": 12,  
    "projectedDropoutsWithoutIntervention": 8,  
    "projectedDropoutsWithIntervention": 3,  
    "preventionRate": 0.625,  
    "intervalLow": 0.45,  
    "intervalHigh": 0.78  
  },  
  "nationalProjection": {  
    "totalMunicipalLinks": 1200,  
    "estimatedAnnualPreventedDropouts": {  
      "low": 8000,  
      "high": 14000  
    }  
  },  
  "citations": [  
    "DSWD 2023 Pantawid Pamilyang Pilipino Program Annual Report",  
    "DepEd Basic Education Statistics 2023",  
    "Reyes & Tabuga (2012). Conditional Cash Transfer Program in the  
Philippines."  
  ]  
}
```

8. Service Layer Specifications

8.1 PredictionService

typescript

```
// lib/services/prediction.ts
```

```
class PredictionService {  
  private model: ONNXModel | null = null;
```

```

async initialize(): Promise<void> {
    // Load ONNX model on server startup
}

async predict(features: ChildFeatures): Promise<Prediction> {
    // For demo: lookup in precomputed_predictions.json
    // For live: run ONNX inference
}

getRiskTier(probability: number): RiskTier {
    if (probability >= 0.65) return 'critical';
    if (probability >= 0.50) return 'high';
    if (probability >= 0.30) return 'moderate';
    return 'low';
}
}

```

8.2 NarrationService

typescript

// lib/services/narration.ts

```

class NarrationService {
    async narrate(
        child: Child,
        prediction: Prediction
    ): Promise<CaseNote> {
        // 1. Check precomputed_narrations.json for childId
        if (DEMO_MODE && this.cache.has(child.id)) {
            return { ...this.cache.get(child.id), source: 'cached' };
        }

        // 2. Build prompt from prediction + features
        const prompt = buildCaseNotePrompt(child, prediction);

        // 3. Call Claude API with structured output
        const response = await anthropic.messages.create({
            model: "claude-sonnet-4-5",
            max_tokens: 400,
            system: CASE_NOTE_SYSTEM_PROMPT,
            messages: [{ role: "user", content: prompt }]
        });

        // 4. Parse, validate, return
        return {

```

```

        childId: child.id,
        narrativeText: response.content[0].text,
        generatedAt: new Date().toISOString(),
        source: 'live'
    };
}
}

```

8.3 LLM Prompt for Case Notes

typescript

```
// lib/prompts.ts
```

```
export const CASE_NOTE_SYSTEM_PROMPT = `
You are writing a case note for a Filipino DSWD social worker
(Municipal Link)
who manages 4Ps families. The note describes one child's current
dropout risk situation.
```

Tone requirements:

- Dignified, never patronizing
- Factual, never alarmist
- Action-oriented, never fatalistic
- Written in English with natural Filipino case-work conventions
- 3 to 5 sentences, no longer

Content requirements:

- Lead with what's happening with this specific child (concrete, not statistical)
- Reference the data driving the concern, in plain language
- End with what the social worker should consider
- Never mention the model, AI, or "the system"
- Never use the words "verified" or "certified" – use "documented" or "observed"
- Never refer to the child's risk as a "label" or "flag" – use "current situation"

Output: A single paragraph. No headers. No markdown.

```

`;

export function buildCaseNotePrompt(
    child: Child,
    prediction: Prediction
): string {
    return `

```


Write a case note for this child:

```
Name: ${child.firstName}
Age: ${child.age}
Grade: ${child.gradeLevel}
Barangay: ${child.barangay}
```

```
Current dropout probability over 90 days:
${(prediction.dropoutProbability90d * 100).toFixed(0)}%
```

```
Top reasons:
${prediction.topRiskDrivers.map(d => `- ${d.humanLabel}`).join("\n")}
```

Write the case note now.

```
` .trim();
}
```

8.4 InterventionService (Rule-Based)

typescript

```
// lib/services/intervention.ts
```

```
const INTERVENTION_RULES: Record<string, Intervention> = {
  attendanceRate30d_dropping: {
    type: 'home_visit',
    urgencyDays: 7,
    description: "Schedule a home visit to understand recent
absences",
    rationale: "Attendance trend reversal is most effective when
addressed early",
    dswdProgramReference: undefined
  },
  householdIncomeShock: {
    type: 'cash_assistance',
    urgencyDays: 14,
    description: "Assess eligibility for DSWD AICS emergency
assistance",
    rationale: "Income shock is a documented dropout precursor",
    dswdProgramReference: "DSWD Assistance to Individuals in Crisis
Situations (AICS)"
  },
  hasOlderSiblingDropout: {
    type: 'family_counseling',
    urgencyDays: 21,
```

```

        description: "Engage family on educational continuity; reference
older sibling's experience",
        rationale: "Sibling dropout patterns indicate household-level
factors",
        dswdProgramReference: "Family Development Sessions (FDS)"
    },
    // ... full mapping
};

class InterventionService {
    recommend(prediction: Prediction): Intervention[] {
        return prediction.topRiskDrivers
            .map(driver => this.mapDriverToIntervention(driver))
            .filter(Boolean)
            .slice(0, 3); // Max 3 to avoid overwhelming the social worker
    }
}

```

8.5 AccessControlService

typescript

```

// lib/services/accessControl.ts

class AccessControlService {
    verify(role: string, resource: string): boolean {
        const scope = ACCESS_SCOPE_CONFIG[role];
        if (!scope) return false;
        return scope.canSee.includes(resource);
    }

    audit(access: AccessAttempt): void {
        // In-memory log for demo; would be persistent in production
        auditLog.push({
            ...access,
            timestamp: new Date().toISOString()
        });
    }
}

```

9. Frontend Specification

9.1 Screen 1 – Landing (/)

Purpose: Demo entry point

Elements:

- PantawidAral wordmark
- Tagline: *"Foresight for the families who can't afford to be invisible."*
- Single CTA: "Login as Ate Marivic Santos"
- Small footer: "Hackathon prototype – synthetic data only"

No real auth. Click → </dashboard>.

9.2 Screen 2 – Dashboard (</dashboard>)

Purpose: Caseload triage view

Layout:

- Header: "Ate Marivic Santos" + "Municipal Link, San Pedro, Laguna" + "247 4Ps families"
- Summary strip: 4 stat cards (Total flagged: 12, Critical: 3, High: 5, Moderate: 4)
- Caseload table: 12 rows, sorted by risk descending
 - Columns: Name | Age | Grade | Risk Tier (color-coded badge) | Probability | Top Concern | Action
 - Click row → [/child/\[id\]](/child/[id])
- Persistent ethics banner at bottom: "🔒 These predictions are visible only to authorized DSWD staff. [See Who Can See This →]"

Interactions:

- Hover row: subtle highlight
- Click row: navigate to detail
- Click ethics banner: navigate to </access>

9.3 Screen 3 – Child Detail ([/child/\[id\]](/child/[id]))

Purpose: The core demo screen – Marivic's understanding of one child

Layout (top to bottom):

1. Header: Child first name + age + grade + barangay
2. Risk panel: Probability gauge (0–100%), risk tier badge, calibrated confidence note
3. Case note card: The LLM-generated paragraph, in a quoted-style frame
4. Why panel: Top 3 risk drivers with human labels and small SHAP-derived bars
5. Recommended interventions: 2–3 cards, each with type, urgency badge, description, DSWD program reference
6. Access log card: "Accessed by: Marivic Santos · 2026-05-07 08:23 · Logged"
7. Action button: "Mark intervention as scheduled" (frontend-only state for demo)

Visual hierarchy: The case note is the focal point. It is the screen's emotional center.

9.4 Screen 4 – Who Can See This (</access>)

Purpose: The Ethics rubric beat

Layout:

- Title: "Who Can See This Information"
- Principle quote (large): *"This system is built to be useful only to those who can help, and useless to those who could harm."*
- Access matrix: 6-row table
 - Role | Can See | Cannot See | Why
 - Color-coded: green for can-see roles, red for cannot-see
- Footnote: Explanation of the labeling effect, with citation to education research

This screen is visually quiet, almost minimalist. The seriousness of the design carries the weight.

9.5 Screen 5 – Impact Projection (/impact)

Purpose: Sustainability + Social Impact rubric beats

Layout:

- Cluster impact: "If interventions are scheduled within 14 days for the 12 flagged children, the modeled outcome is 5 prevented dropouts in San Pedro this year."
- National projection chart: Bar chart showing 1,200 Municipal Links × intervention rate × prevention rate = 8,000–14,000 annually
- Citations: Three labeled sources (DSWD, DepEd, academic research)
- Roadmap line: "Pilot path: 90-day evaluation with one DSWD field office, co-designed with DSWD National Advisory Committee."

9.6 Component Notes

- `<RiskBadge tier={tier} />`: Returns a styled badge – gray (low), yellow (moderate), orange (high), red (critical)
- `<EthicsBanner />`: A persistent footer component shown across all screens reinforcing access scope
- `<AccessScopeMatrix />`: The core component of `/access`; receives the matrix from API

10. Hour-by-Hour Build Plan

Hour	Goal	Deliverable
0 (pre-hackathon)	Run all <code>ml/</code> scripts: generate dataset, train model, compute SHAP, pre-generate narrations, fairness audit	All artifacts in <code>/data</code>

1	Project skeleton, types, static data files loaded, mock API routes returning stubs	All API routes return valid stub data
2	Real <code>PredictionService</code> (loading from precomputed JSON) + Real <code>/api/caseload</code> and <code>/api/child</code>	Dashboard renders 12 real children
3	<code>NarrationService</code> integrated; live LLM call with cache fallback; child detail page renders case note	Click any child → see full case note
4	<code>InterventionService</code> rules + <code>/api/access-scope</code> + Access Scope screen	Ethics screen complete
5	Impact projection screen + visual polish + responsive check + caseload sorting/styling	Full demo flow runs cleanly
6	Demo rehearsal, fallback testing (LLM offline → cached works), deploy to Vercel, final smoke test	Shippable URL on demo machine

Cut order if behind:

1. First cut: Live LLM mode (use only cached narrations)
2. Second cut: National projection chart (replace with text)
3. Third cut: Fairness audit chart (mention verbally only)
4. Never cut: Access Scope screen or the case note rendering

11. Failure Modes & Mitigation

Failure	Mitigation
LLM API down during demo	Pre-cached narrations for 3 demo children; demo proceeds offline

ONNX model fails to load	Fall back to Python subprocess inference; or pre-computed predictions only
Vercel deploy fails	Local backup running via <code>npm run start</code> on demo machine
Network failure during demo	Pre-record 30-second screen capture as ultimate fallback
Judge clicks a non-demo child	All 247 children have valid (faster, less polished) auto-generated narrations
LLM returns unexpected format	Schema validation catches; fall back to template-based case note

12. Demo Persona Specification

Three children must be hand-tuned for the pitch:

Mark Aquino, 14, Grade 8

- Probability: 0.73 (Critical)
- Top drivers: Attendance crash (96% → 71%), parental income shock, older sibling dropped out 2024
- Story: Father lost construction work; Mark started missing school to help with sari-sari store; sister Cristine left in Grade 9
- Pre-cached case note: Hand-edited for emotional resonance and accuracy

Sofia Reyes, 11, Grade 5

- Probability: 0.58 (High)
- Top drivers: Academic decline (84 → 71 average), family relocation 4 months ago, new school adjustment
- Story: Family moved from Cavite to Laguna for father's new job; Sofia struggling to adapt; teacher hasn't connected with parents

Joshua de la Cruz, 16, Grade 10

- Probability: 0.41 (Moderate)
- Top drivers: Subtle attendance dip, missing 2 of 4 quarterly assessments, age-grade mismatch (held back once)
- Story: Slow drift; the kind of case that historically got missed until too late; the "subtle catch" that proves the model's value

These three are explicitly the demo path. Everything else is supporting context.

13. Pitch-Aligned Copy (Lock These Strings)

These exact strings must appear in the UI:

- App name: "PantawidAral"
 - Tagline: "Foresight for the families who can't afford to be invisible."
 - Ethics principle: "Built to be useful only to those who can help, and useless to those who could harm."
 - Risk score language: "current situation" — never "label," "flag," or "tag"
 - Confidence language: "documented" or "observed" — never "verified" or "certified"
 - Persona role: "Municipal Link" (the official DSWD title)
 - Access banner: "🔒 Visible only to authorized DSWD staff."
-

14. Out of Scope — Will Not Build

If suggested mid-build, the answer is "roadmap, not prototype":

- User authentication, login, multi-user
 - Real DepEd / DSWD data integration
 - Mobile app
 - Notification system (email, SMS, push)
 - Tagalog UI translation (English only for prototype)
 - Family-facing consent flow UI
 - Persistent intervention tracking
 - Predictive maintenance / model retraining loop
 - Multi-cluster / multi-municipality view
 - DSWD admin / supervisor view
-

15. Done Definition

The hackathon build is complete when:

1. The full demo flow runs end-to-end with no manual interventions
 2. All three demo children render with full case notes (live or cached)
 3. The Access Scope screen is visually polished and accurate
 4. The Impact Projection page cites real sources
 5. The deployed Vercel URL works from the demo machine
 6. Fallbacks for LLM failure and network failure have been tested
 7. The pitch script has been rehearsed at least twice end-to-end
 8. The synthetic dataset documentation is included in the README
-

END OF SDD

This document is the binding technical specification for the PantawidAral hackathon build. Every section above answers an implementation question and protects against scope creep. When in doubt during the build, return to this document.